Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

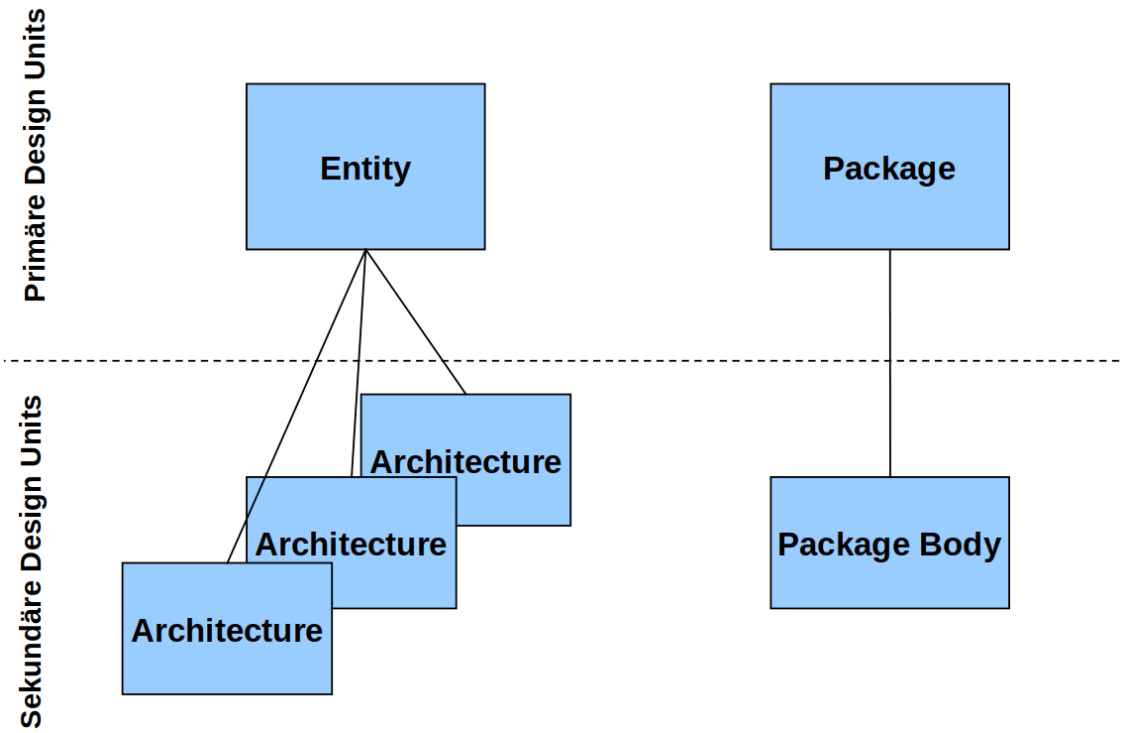
Hardware-Software-Systeme

Kapitel 03 - VHDL - Hardware-Beschreibungssprache

Prof. Dr.-Ing. Guillermo Payá Vayá, 28. Oktober 2025

Der Aufbau von VHDL-Systemen

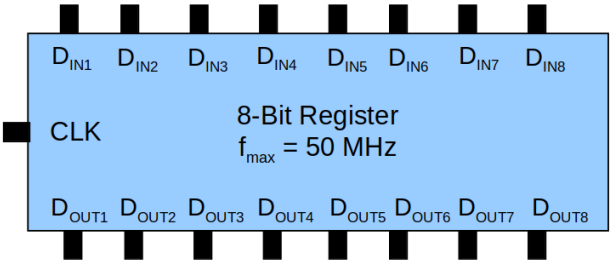
Der Aufbau von VHDL-Systemen - Design Units



Design Unit: Entity

Eine *Entity* besteht aus

- **Parametern**, welche die Systemstruktur betreffen (z.B. Breite eines Busses, max. Takt-Frequenz)
- **Verbindungen**, die Informationen in das System und aus dem System transferieren




```
entity n_bit_register is
  generic (
    width : integer := 8;
    fmax  : integer := 50
  );
  port (
    D_IN  : IN bit_vector(0 to width-1);
    D_OUT:OUT bit_vector(0 to width-1);
    CLK   : IN bit
  );
end entity n_bit_register;
```

Design Unit: Architecture

Eine *Entity* (z.B. x86 Prozessor) kann mehrere *Architectures* besitzen (z.B. AMD, INTEL, CYRIX), die alle dasselbe Interface besitzen

```
entity x86 is
  port(
        ...
      );
end entity;
```

```
architecture AMD of x86 is
          ...
end architecture AMD;
```

```
architecture CYRIX of x86 is
          ...
end architecture CYRIX;
```

```
architecture INTEL of x86 is
          ...
end architecture INTEL;
```

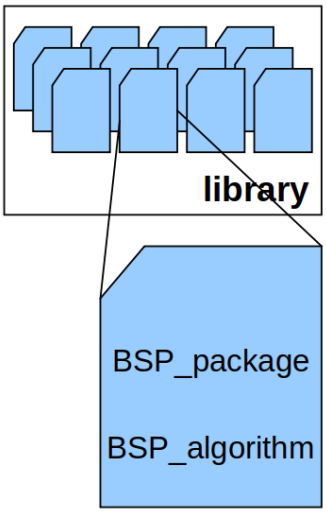

Design Unit: Package

Packages enthalten Elemente, z.B. Funktionen, Typen- und Konstantendeklarationen, *Entities*, etc., die nicht zum VHDL-Standard gehören.

```
library BSP_Lib;
use BSP_lib.BSP_package.BSP_algorithm;

entity Beispiel is
        ...
end entity Beispiel;

architecture a of Beispiel is
        ....
        BSP_algorithm
end architecture a;
```



Vordefinierte Packages: Beispiele

STD_LOGIC_1164

- Erweiterung des VHDL-Standards um IEEE Standard 1164
- Wichtigstes Element: Multi-Value-Logic (std_logic)

```
library IEEE;
use IEEE.std_logic_1164.all;
```

NUMERIC_STD

- Deklaration von Vektoren von std_logic
  - vorzeichenlos (unsigned)
  - vorzeichenbehaftet (signed)
- Definition von arithmetischen, logischen und Schiebeoperatoren
- Definition von Funktionen für die Datentypkonvertierung

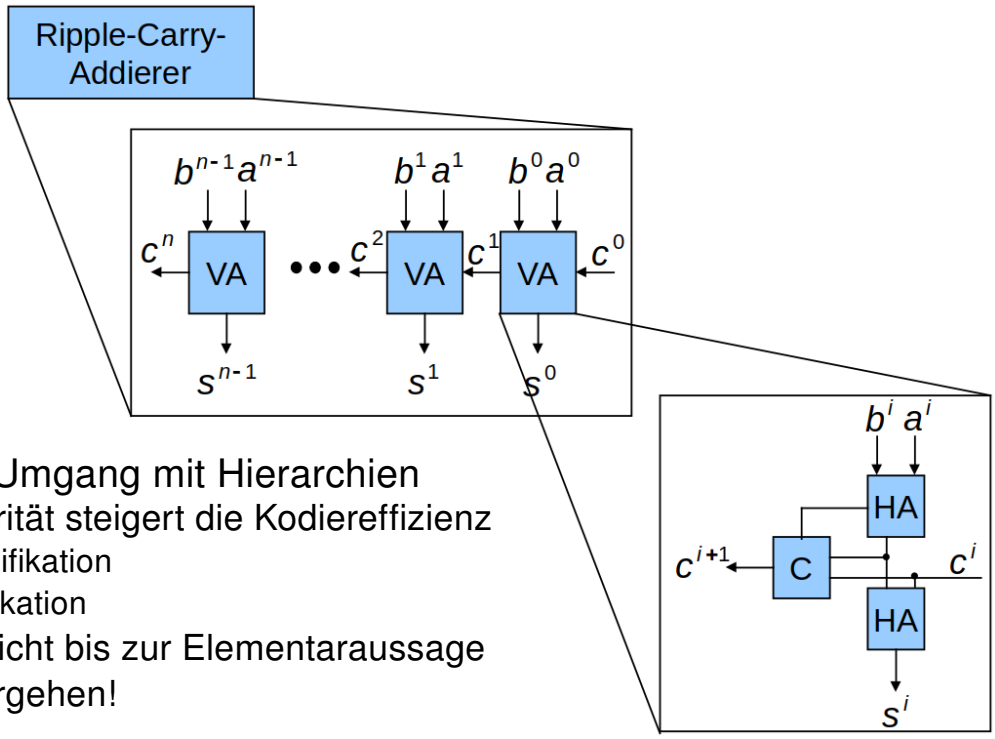
```
library IEEE;
use IEEE.numeric_std.all;
```


Funktionale vs. strukturelle Beschreibung

- **Funktionale Beschreibung** (*behavioral description*):
  Was soll das System machen?
- **Strukturelle Beschreibung** (*structural description*):
  Wie soll diese Funktion erreicht werden?
  Wie ist die Struktur des Systems?

- *Coding for synthesis vs. coding for simulation*
  - Einige Elemente von VHDL sind nicht synthetisierbar
  - Eine Beschreibungsart, die für eine effiziente Simulation sorgt, mag zu einer ineffizienten HW-Architektur führen
  - Komplexe Hierarchien in strukturellen Beschreibungen mögen in die Hardware resultieren, die gewünscht wird, stellen allerdings hohe Ansprüchen an die Verifikationsumgebung
    $\rightarrow$Trade-off zwischen Granularität und Verifikationsgeschwindigkeit

Hierarchie in VHDL-Beschreibungen



- Richtiger Umgang mit Hierarchien
  - Modularität steigert die Kodiereffizienz
    - Spezifikation
    - Verifikation
  - Aber: Nicht bis zur Elementaraussage hinuntergehen!

Datentypen, Signale, Konstanten und Attribute

Datentypen in VHDL: skalare Standardtypen (I)

- VHDL ist eine stark typisierte Sprache!!
  - Keine implizite Datentypkonvertierung
  - Keine automatische Breitenanpassung
- Skalare (& synthetisierbare) Typen
  - `boolean`: false & true
  - `integer`
    - Rang: -2^{31} bis $2^{31} - 1$
    - Dezimal ist Standard, sonst wird das Zahlenformat angegeben. Zum Beispiel: `16#cafe#`, `2#011101#`
  - `bit`: die beiden logischen Werte '0' & '1'
  - Aufzählungstypen: für eigens definierte Typen
    - unabhängig von technischen Codierungen
    - erhöhen die Lesbarkeit der Beschreibung

```
type wetter_t is (REGEN, SONNE, NEBEL);
type fsm_state_t is (IDLE, RUN, WAIT, COPY);
```


Datentypen in VHDL: skalare Standardtypen (II)

- Skalare Typen, die ohne Weiteres **nicht synthetisierbar** sind
  - `character`
    - Entspricht dem ISO 8859-1 Zeichensatz
    - Die Zeichen werden in Hochkommas eingeschlossen, 'a',..,'z'
  - `real` (Fließkommazahlen)
    - -1.0e+38 bis +1.0e38
  - physikalische Maßeinheiten (z.B. `time`)
    - Die Einheiten fungieren als Umrechnungsfaktoren
    - Modellierung von Verzögerungen. Bsp: `c <= ... after 2ns;`
  - Untertypen, die ebenfalls zum Standard gehören
    - `positive`, alle integer von 1 bis $2^{31} - 1$
    - `natural`, alle von 0 bis $2^{31} - 1$
  - Andere nicht-synthetisierbare Typen
    - `access`: Zeiger-typ (wird oft für die Verhaltensmodellierung größerer Speicher verwendet)
    - `file`: für I/O Zwecke in Testbenches

Datentypen in VHDL: Multi-Value logic

- Der IEEE 1164 Standard ist im externen package std_logic_114 definert und muss explizit eingebunden werden

```
library IEEE;
use IEEE.std_logic_1164.all;
```

- Die Datentypen `std_logic` und `std_ulogic` enthalten mehr Zustände als der Standard-Typ `bit`, um Konflikte durch mehrere Treiber darzustellen und aufzulösen.
  - Auflösungsfunktionen gibt es nur bei `std_logic`!!
- Für Synthese und Simulation besser geeignet als der Typ `bit`.

```
TYPE std_logic is
 ('U', -- Uninitialized
  'X', -- 0/1 ? (Konflikt)
  '0', -- 0
  '1', -- 1
  'Z', -- hohe Impedanz
  'W', -- L/H? (Konflikt)
  'L', -- schwache 0
       -- (f\"ur Pull-Down)
  'H', -- schwache 1
       -- (f\"ur Pull-Up)
  '-'  -- don't care
 );
```


Datentypen in VHDL: komplexe Typen (Arrays)

- Komplexe Datentypen, die eine reguläre Struktur haben. Diese bestehen aus Elementen des gleichen Datentyps.
- Die Größe eines `Arrays` wird durch ein Intervall (`range`) bestimmt. Hierbei muss auf die Richtung (`to/downto`) geachtet werden!

```
type vierzahlen is array (3 downto 0) of integer; -- beschränkt
type bit_vector is array (natural range <>) of bit; -- unbeschränkt
```

```
-- Bei unbeschränkten Intervallen muss bei Deklaration des Objektes oder
-- des Untertyps das konkrete Intervall definiert werden!
subtype dreibits is bit_vector(0 to 2);
```

- Mehrdimensionale Arrays sind möglich, aber nur bedingt empfehlenswert (Code-Lesbarkeit, Simulation- & Syntheseeffizienz)
- Anstelle des Standard-Typen, `string` und `bit_vector`, werden für die HW-Beschreibung die Typen `std_logic_vector` und `signed/unsigned` verwendet (`array of std_logic`)

Datentypen in VHDL: komplexe Typen (Records)

- Records fassen Elemente unterschiedlicher Typen zusammen
  - Skalare
  - Eigene Typen oder Untertypen (auch komplexe)
- Hauptnutzung: Bildung abstrakter Datenmodelle

```
type MEM_Stat_t is
     record
        VALID      : boolean;
        START_ADDR : std_ulogic_vector(7 downto 0);
        N_OF_WORDS : integer range 0 to 2*BUS_WIDTH_C-1;
     end record;
```

- Die Dereferenzierung erfolgt über die Namen der einzelnen Felder

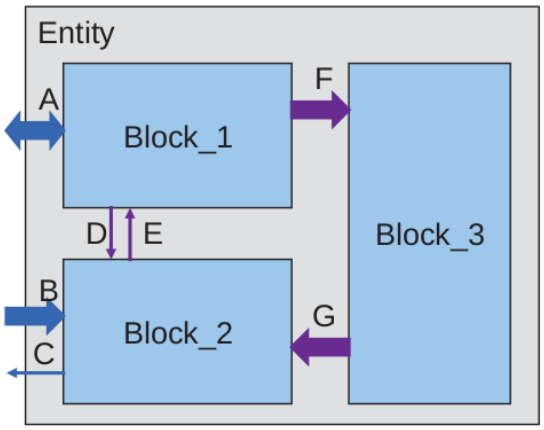
```
-- Bei einem Ojekt (z.B. RAM_st) des Typs MEM_Stat_t
RAM_st.VALID      := true;
RAM_St.START_ADDR  := "00010000";
RAM_St.N_OF_WORDS := 0;
```


Bezeichner (`identifier`)

- `Identifier` ist der Name von Signalen, Entities, Konstanten, etc.
- Regeln
  - Ein `Identifier` muss in eine Zeile passen
  - Ein `Identifier` muss mit einem Buchstaben anfangen
  - Ein `Identifier` kann aus Buchstaben, Zahlen und Unterstrichen bestehen
  - Es dürfen nicht zwei Unterstriche hintereinander, bzw. Unterstriche am Anfang oder Ende eines `Identifier` stehen.
  - Ein `Identifier` darf keine Leerzeichen enthalten
  - `Identifiers` sind nicht case-sensitive
  - Reservierte Worte dürfen nicht als `Identifier` verwendet werden
- Beispiele für gültige `Identifier`
  - `multiplex32_nxt`
  - `MEM_ADDR_WIDTH_C`

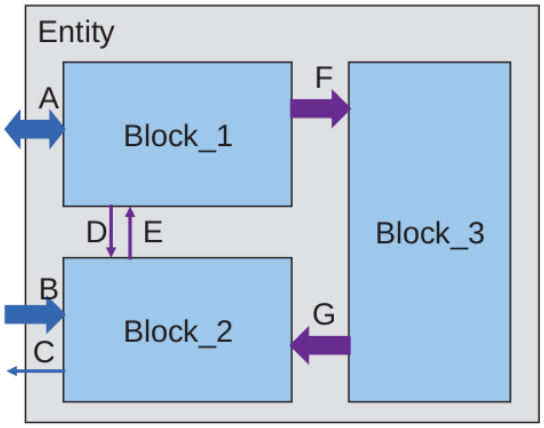
Signale in VHDL: Varianten

- Zwei Varianten abhängig von ihrer Funktion
  - Anbindungen mit der „Umwelt“ werden als `port` deklariert (im Bsp. A, B, C)
  - Anbindungen zwischen internen Blöcken werden als `signal` deklariert (im Bsp. D, E, F, G)
- Signale können von jedem Type sein
  - Für `ports` werden allerdings die Typen std_(u)logic bzw. std_(u)logic_vector empfohlen



Signale in VHDL: `ports`

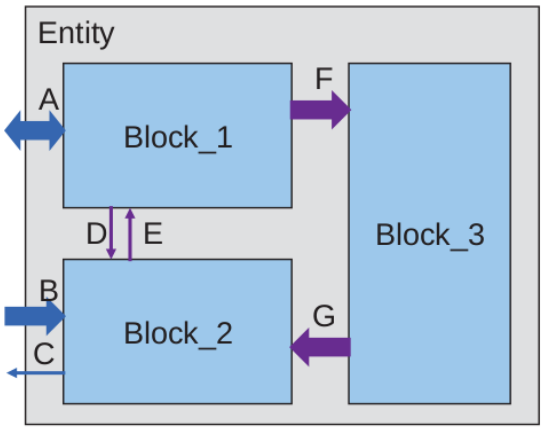
- Die sog. `ports` werden in der `Entity` deklariert
- In die Deklaration gehören
  - der Name (`identifier`)
  - die Richtung (`mode`)
  - der Typ
- Achtung: Nichts in einer Entity darf direkt aus einem ihrer Ausgangsports lesen!!
  $\rightarrow$ Hilfssignal erforderlich




```
entity Beispiel is
  port (
    A : inout std_ulogic_vector(7 downto 0);
    B : in    std_ulogic_vector(7 downto 0);
    C : out   std_ulogic);
end Beispiel;
```

Signale in VHDL: interne Signale

- Interne Signale dienen zur Kommunikation zwischen Blöcken einer Architektur
- Die Deklaration eines Signals erfolgt in einer `architecture`, einem Block oder einem Unterprogramm

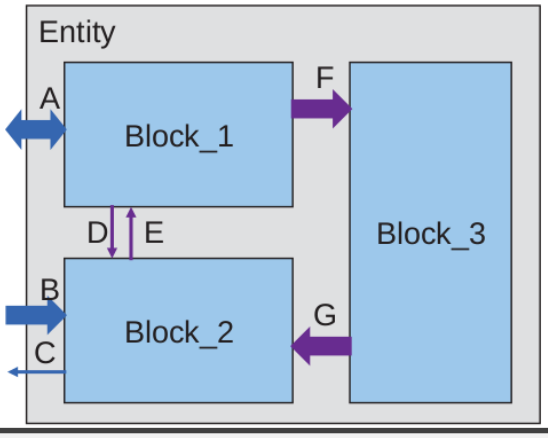



```
signal <name>: <datentyp>;
```

```
architecture arch_1 of Beispiel is
  signal F, G : std_ulogic_vector(7 downto 0);
  signal D, E : std_ulogic;
begin -- hier fangen die Anweisungen der Architektur an
  ...
end arch_1;
```


Hierarchie in VHDL: Einführung

- Um Einheiten in einer Architektur einbinden zu können, muss es einen Platzhalter für sie geben → ein `component` wird hierfür verwendet.




```
component component_name
  generic (generic_list);
  port (port_list);
end component;
```

```
architecture arch_1 of Beispiel is
  component Block_3
    port (COEFF : in  std_ulogic_vector(15 downto 0);
          RES   : out std_ulogic_vector(15 downto 0));
    ... ; -- Deklaration von Signalen und mehr Komponenten
begin
  U1 : Block_3 port map (COEFF => F, RES => G);
  ...
end arch_1;
```

Signale: Sichtbarkeit

- Die Sichtbarkeit von Signalen hängt von dem Ort der Deklaration ab
  - Ein `signal`, das als `port` in einer `entity` deklariert wurde, ist in allen `architectures`, die der `entity` zugeordnet sind, sichtbar
  - Ein `signal`, das in einer `architecture` deklariert wird, ist nur in dieser `architecture` sichtbar.
  - Ein `signal`, das in einem Block einer `architecture` deklariert wird, ist nur in diesem Block sichtbar.
  - Ein `signal`, das in einem `package` deklariert wurde, ist in allen Design-Units sichtbar, die dieses `package` benutzen.
    - Solche globale Signale sind nicht synthetisierbar, sondern eignen sich nur für die rein funktionale Modellierung!!
- Für die HW-Beschreibung wird empfohlen, Signale **nur** im entsprechenden Abschnitt einer `architecture` zu deklarieren! Ausnahmsweise können Signale in Funktionen (Unterprogramme) deklariert und verwendet werden.

Signale: Zuweisungen

- Einem Signal kann ein anderes Signal, eine Variable oder ein fester Wert (Konstante) zugewiesen werden
- Signalzuweisungen erfolgen durch das Symbol <=
- Typ und Arraybreite müssen auf beiden Seiten zueinander passen
  - Keine automatische Typkonvertierung
  - Keine automatische Anpassung der Vektorlänge

```
  signal a, b : std_ulogic_vector(7 downto 0);
  signal c    : integer range 0 to 255;
begin
  a <= b;
  a <= "00100111"; -- alternativ auch a <= x"27";
  b(7 downto 4) <= a(7 downto 4);
  c <= 32;
  b(0) <= '1';
  a <= (others => '1');
  b <= a(3 downto 0) & a(7 downto 4);
```

Konstanten

- Die Werte von Konstanten (`constant`) werden bei der Übersetzung des VHDL-Codes seitens des Synthese-Tools festgelegt
- Vorteile
  - Erhöhung der Lesbarkeit des Codes
  - Module sind portabler für andere Projekte, da sie leichter angepasst werden können
- Konstanten können in einem `package`, einer `architecture`, einem Block (`procedure/function`) oder einem `process` deklariert werden
- Mögliche Funktionen für Konstanten
  - Spezifikation der Größe von komplexen Objekten
  - Kontrolle von Loop-Zählern
  - Definition von Modul-Parametern

```
constant loopNumber : integer := 4;
```


Attribute (I)

- Attribute werden dafür benutzt, Informationen aus Signalen, Typen oder anderen Objekten verwenden zu können
- Zwei Klassen von attributen: „1076 Standard“ und „Custom“

```
-- standard attributes
type bit_array is array (1 to 5) of bit;
signal L : integer := bit_array'left; -- L has a value of 1

type bit_array is array (15 downto 0) of bit;
signal I : integer := bit_array'right; -- I has the value of 0

type state_type is (INIT, HOLD, STROBE, READ, IDLE);
signal F : integer := state_type'pos(READ); -- P hast the value of 3;

-- custom attribute
attribute ram_style : string;
attribute ram_style of MEMORY : signal is mem_style_g;
```

Attribute (II)

T is an enumeration, integer, floating or physical type or subtype	
T'BASE	is the base type of the type T
T'LEFT	is the leftmost value of type T. (Largest if downto)
T'RIGHT	is the rightmost value of type T. (Smallest if downto)
T'HIGH	is the highest value of type T.
T'LOW	is the lowest value of type T.
T'ASCENDING	is boolean true if range of T defined with to .
T'IMAGE(X)	is a string representation of X that is of type T.
T'VALUE(X)	is a value of type T converted from the string X.
T'POS(X)	is the integer position of X in the discrete type T.
T'VAL(X)	is the value of discrete type T at integer position X.
T'SUCC(X)	is the value of discrete type T that is the successor of X.
T'PRED(X)	is the value of discrete type T that is the predecessor of X.
T'LEFTOF(X)	is the value of discrete type T that is left of X.
T'RIGHTOF(X)	is the value of discrete type T that is right of X.

Attribute (III)

A is an array signal, variable, constant, type or subtype	
A'LEFT	is the leftmost subscript of array A or constrained array type.
A'LEFT(N)	is the leftmost subscript of dimension N of array A.
A'RIGHT	is the rightmost subscript of array A or constrained array type.
A'RIGHT(N)	is the rightmost subscript of dimension N of array A.
A'HIGH	is the highest subscript of array A or constrained array type.
A'HIGH(N)	is the highest subscript of dimension N of array A.
A'LOW	is the lowest subscript of array A or constrained array type.
A'LOW(N)	is the lowest subscript of dimension N of array A.
A'RANGE	is the range A'LEFT to A'RIGHT or A'LEFT downto A'RIGHT .
A'RANGE(N)	is the range of dimension N of A.
A'REVERSE_RANGE	is the range of A with to and downto reversed.
A'REVERSE_RANGE(N)	is the REVERSE_RANGE of dimension N of array A.
A'LENGTH	is the integer value of the number of elements in array A.
A'LENGTH(N)	is the number of elements of dimension N of array A.
A'ASCENDING	is boolean true if range of A defined with to .
A'ASCENDING(N)	is boolean true if dimension N of array A defined with to .

Attribute (IV)

S is a signal	
S'DELAYED(t)	is the signal value of S at time now - t .
S'STABLE	is true if no event is occurring on signal S.
S'STABLE(t)	is true if no even has occurred on signal S for t units of time.
S'QUIET	is true if signal S is quiet. (no event this simulation cycle)
S'QUIET(t)	is true if signal S has been quiet for t units of time.
S'TRANSACTION	is a bit signal, the inverse of previous value each cycle S is active.
S'EVENT	is true if signal S has had an event this simulation cycle.
S'ACTIVE	is true if signal S is active during current simulation cycle.
S'LAST_EVENT	is the time since the last event on signal S.
S'LAST_ACTIVE	is the time since signal S was last active.
S'LAST_VALUE	is the previous value of signal S.
S'DRIVING	is false only if the current driver of S is a null transaction.
S'DRIVING_VALUE	is the current driving value of signal S.

Beispiel Signal und Attribute

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.